

# Documentacion Microservidor History

## Introducción

El microservicio History pertenece al sistema distribuido de alerta ciudadana tipo C5.

Su función principal es administrar el registro histórico de todas las alertas generadas dentro del sistema, permitiendo almacenar, consultar y mantener evidencia de los incidentes ocurridos.

Este servicio trabaja de manera independiente siguiendo la arquitectura de microservicios, donde cada componente tiene una responsabilidad específica.

En lugar de que todos los servicios guarden información directamente, el sistema delega la persistencia al microservicio History.

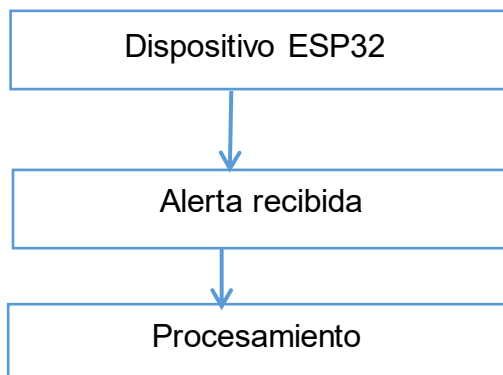
## Objetivo del microservicio

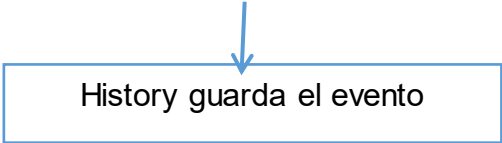
El objetivo de History es garantizar que ninguna alerta importante se pierda y que exista un registro permanente para:

- Auditorías.
- Consultas posteriores.
- Análisis de incidentes.
- Seguimiento de emergencias.
- Generación de estadísticas.

Ejemplo:

Un usuario presiona un botón de emergencia:



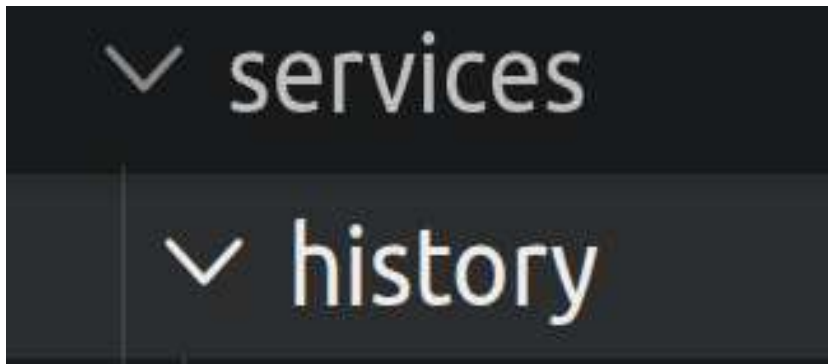


History guarda el evento

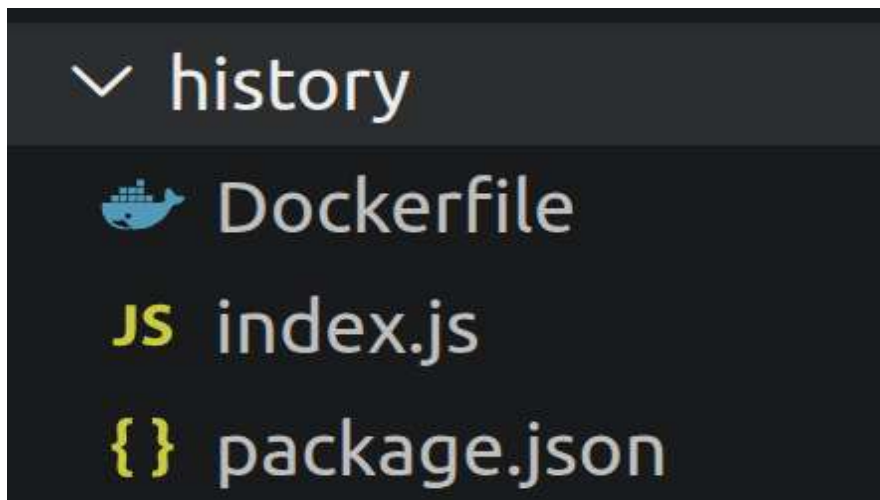
Después se puede consultar: Qué emergencias ocurrieron hoy y el sistema puede responder con los registros almacenados.

### Estructura del microservicio

La carpeta:



contiene:



Cada archivo tiene una función específica.

#### Archivo Dockerfile

El Dockerfile permite crear la imagen del servicio.

Su función es preparar el ambiente donde correrá History.

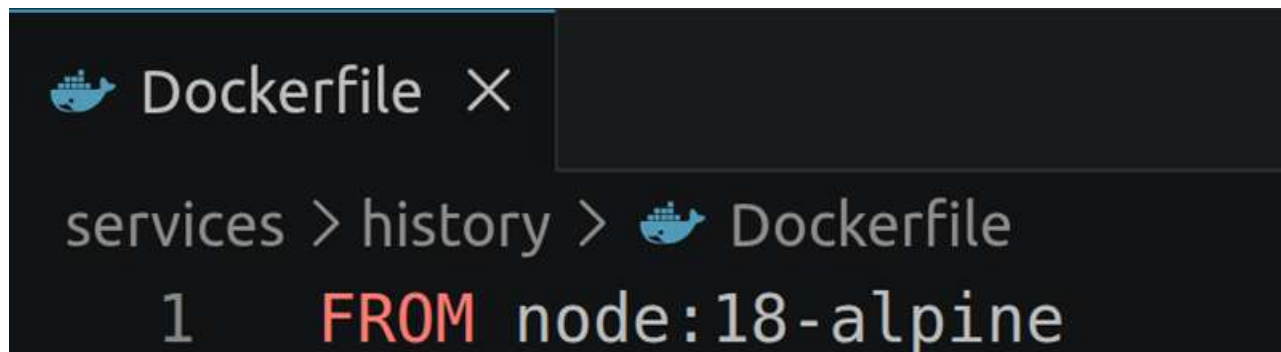
Normalmente realiza:

1. Seleccionar una imagen base de Node.js.

Ejemplo:

**FROM node**

Esto indica que el servicio se ejecutará usando Node.

A screenshot of a code editor window titled 'Dockerfile' with a close button. The breadcrumb navigation shows 'services > history > Dockerfile'. The first line of the Dockerfile is '1 FROM node:18-alpine', where 'FROM' is highlighted in red.

```
services > history > Dockerfile
1  FROM node:18-alpine
```

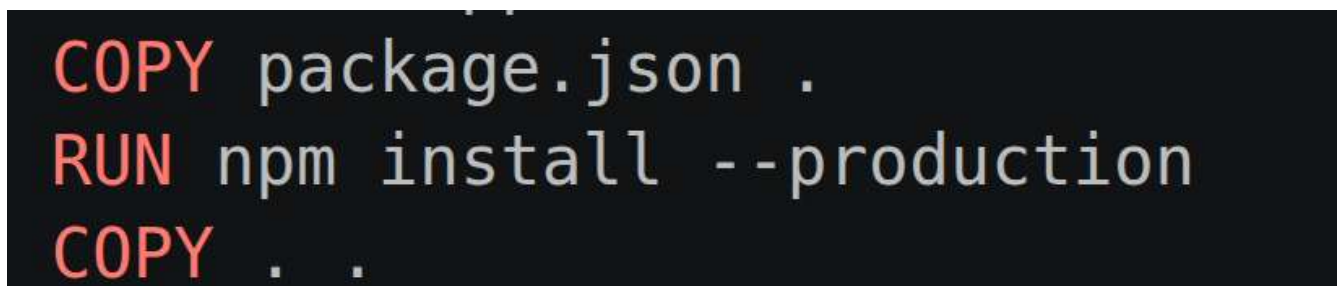
2. Copiar los archivos del proyecto:

**COPY .**

Copia:

- index.js
- package.json

dentro del contenedor.

A screenshot of a code editor showing three lines of a Dockerfile. The first line is 'COPY package.json .' with 'COPY' in red. The second line is 'RUN npm install --production' with 'RUN' in red. The third line is 'COPY . .' with 'COPY' in red.

```
COPY package.json .
RUN npm install --production
COPY . .
```

3. Instalar dependencias:

**npm install**

Instala todas las librerías necesarias.

```
RUN npm install --production
```

4. Ejecutar el servicio:

**node index.js**

Cuando Docker levanta el contenedor, automáticamente inicia History.

```
Dockerfile ×
services > history > Dockerfile
1 FROM node:18-alpine
2 WORKDIR /app
3 COPY package.json .
4 RUN npm install --production
5 COPY . .
6 CMD ["node", "index.js"]
7
```

### Archivo package.json

Este archivo administra la configuración del proyecto.

Contiene:

#### Nombre

Identifica el servicio.

```
"name": "c5-history",
```

## Dependencias

Son librerías necesarias para funcionar.

**express:** Sirve para crear la API.

**Scripts:** Permiten ejecutar comandos.

Ejemplo:

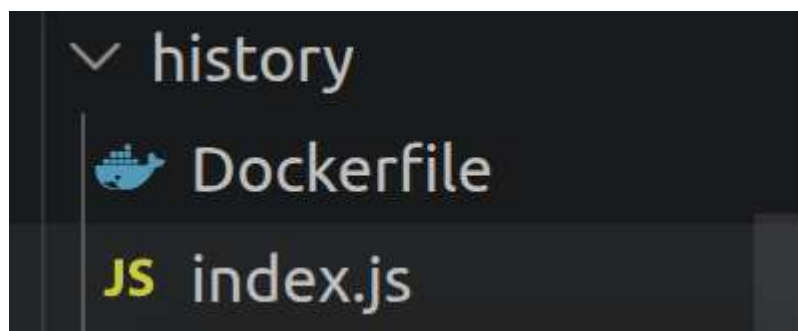
**npm start**

Ejecuta:

node index.js

```
Dockerfile {} package.json X
services > history > {} package.json > {} dependencies > abc pg
1  {
2    "name": "c5-history",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": { "start": "node index.js" },
6    "dependencies": {
7      "express": "^4.18.2",
8      "ioredis": "^5.3.2",
9      "pg": "^8.11.3",
10     "mqtt": "^5.3.4"
11   }
12 }
```

Archivo principal index.js



Este es el cerebro del microservicio.

Aquí está la lógica.

Sus responsabilidades son:

- Crear servidor.
- Recibir solicitudes.
- Procesar alertas.
- Responder consultas.

### **Inicio del servidor**

Cuando inicia:

```
node index.js
```

el servicio crea un servidor HTTP.

Ejemplo:

History Service iniciado

Puerto: 5005

Esto significa que está listo para recibir información.

### **Recepción de una alerta**

Cuando llega una alerta:

Ejemplo:

```
{  
  "id":"A001",  
  "device":"ESP32-01",  
  "tipo":"incendio",  
  "lat":19.70,  
  "lon":-99.50,  
  "prioridad":"critica",  
  "fecha":"2026-06-16"  
}
```

History recibe esa información.

Primero valida:

- Si tiene ID
- Si tiene fecha
- Si tiene ubicación
- Si tiene prioridad

Si todo está correcto continúa.

## Almacenamiento

Después de validar:



La información queda guardada.

Ejemplo:

Tabla:

| id  | tipo     | Prioridad | zona |
|-----|----------|-----------|------|
| 001 | incendio | critica   | lab  |

## Consulta del historial

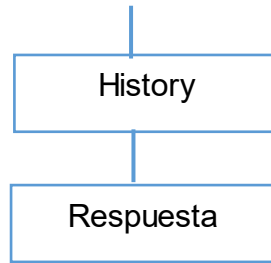
El operador puede pedir:

Muéstrame las alertas crítica

History consulta:

Base de datos





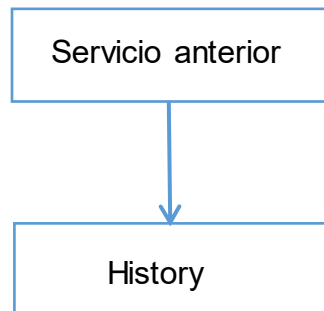
Devuelve:

```
[  
{  
  "tipo":"incendio",  
  "prioridad":"critica"  
}]
```

### Comunicación con otros servicios

History no depende directamente del ESP32.

La comunicación es:



Esto permite que si cambia el ESP32 no sea necesario modificar History.

### Código de Index.js

```
const express = require("express");  
const { Pool } = require("pg");  
const Redis = require("ioredis");  
const https = require("https");
```

```

const http = require("http");
const mqtt = require("mqtt");

const app = express();
app.use(express.json({ limit: "10mb" }));

app.use((req, res, next) => {
  res.header("Access-Control-Allow-Origin", "*");
  res.header("Access-Control-Allow-Methods", "GET, POST, PATCH, OPTIONS");
  res.header("Access-Control-Allow-Headers", "Origin, X-Requested-With, Content-Type, Accept");
  if (req.method === "OPTIONS") return res.sendStatus(200);
  next();
});

const PORT = process.env.PORT || 3005;
const REDIS_URL = process.env.REDIS_URL || "redis://localhost:6379";
const MQTT_URL = process.env.MQTT_BROKER || "mqtt://mosquitto:1883";

// — Mapa de cámaras por tótem/zona del campus —
// Usar IP:puerto del stream MJPEG del ESP32-CAM o cámara IP
const CAMARAS = {
  "ESP32-001": "http://192.168.10.101:8080",
  "ESP32-002": "http://192.168.10.102:8080",
  "default": "http://192.168.10.101:8080",
};

function getCamURL(device_id) {
  return CAMARAS[device_id] || CAMARAS["default"];
}

const masterPool = new Pool({ connectionString: process.env.DB_MASTER });
const replicaPool = new Pool({ connectionString: process.env.DB_REPLICA });

masterPool.on("connect", () => console.log("[History] Conectado a PostgreSQL MASTER"));

```

```

replicaPool.on("connect", () => console.log("[History] Conectado a PostgreSQL RÉPLICA"));

const redis = new Redis(REDIS_URL);

// — MQTT para feedback a los tótems —

let mqttClient;

function conectarMQTT() {
  mqttClient = mqtt.connect(MQTT_URL, { clientId: "c5_history_campus" });
  mqttClient.on("connect", () => console.log("[History] MQTT conectado"));
  mqttClient.on("error", (err) => console.error("[History] MQTT error:", err.message));
}

// Notificar al tótem ESP32 el estado de su alerta
function notificarTotem(device_id, status) {
  if (!mqttClient || !mqttClient.connected) {
    console.warn("[History] MQTT no conectado — no se pudo notificar al tótem");
    return;
  }
  // Soportar ambos prefijos: campus/ y c5/
  const topics = [
    `campus/alerts/status/${device_id}`,
    `c5/alerts/status/${device_id}`
  ];
  const payload = JSON.stringify({ status, campus: true });
  topics.forEach(topic => {
    mqttClient.publish(topic, payload, { qos: 1 }, (err) => {
      if (!err) console.log(`[History] Tótem ${device_id} notificado → ${status} (${topic})`);
    });
  });
}

// — Capturar snapshot —

```

```

async function capturarSnapshot(device_id) {
  const camURL = getCamURL(device_id);
  const url = `${camURL}/shot.jpg`;
  return new Promise((resolve) => {
    const client = url.startsWith("https") ? https : http;
    const req = client.get(url, { timeout: 5000 }, (res) => {
      if (res.statusCode !== 200) { return resolve(null); }
      const chunks = [];
      res.on("data", (chunk) => chunks.push(chunk));
      res.on("end", () => {
        resolve(`data:image/jpeg;base64,${Buffer.concat(chunks).toString("base64")}`);
      });
      req.on("error", () => resolve(null));
      req.on("timeout", () => { req.destroy(); resolve(null); });
    });
  });
}

```

```

// ——— Migración de BD ———
async function migrateDB() {
  try {
    await masterPool.query(`
      ALTER TABLE alerts ADD COLUMN IF NOT EXISTS snapshot_url TEXT;
      ALTER TABLE alerts ADD COLUMN IF NOT EXISTS transcription TEXT;
      ALTER TABLE alerts ADD COLUMN IF NOT EXISTS location_name TEXT;
      ALTER TABLE alerts ADD COLUMN IF NOT EXISTS guard_post TEXT;
      ALTER TABLE alerts ADD COLUMN IF NOT EXISTS zone_description TEXT;
    `);
    console.log("[History] Columnas de campus listas");
  } catch (err) {
    console.error("[History] Error en migración:", err.message);
  }
}

```

```
// — Procesador de cola de alertas —

async function processQueue() {
  console.log("[History] Escuchando cola queue:history...");
  while (true) {
    try {
      const result = await redis.brpop("queue:history", 5);
      if (!result) continue;
      const alert = JSON.parse(result[1]);

      console.log(`[History] Guardando alerta de ${alert.location_name} || ${alert.device_id}...`);
      const snapshot = await capturarSnapshot(alert.device_id);

      await masterPool.query(
        `INSERT INTO alerts
        (id, device_id, latitude, longitude, emergency_type, priority, zone, status, timestamp,
        snapshot_url, location_name, guard_post, zone_description)
        VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, $13)
        ON CONFLICT (id) DO UPDATE SET snapshot_url = EXCLUDED.snapshot_url`,
        [
          alert.alert_id || `ALT-${Date.now()}`,
          alert.device_id,
          alert.latitude,
          alert.longitude,
          alert.emergency_type,
          alert.priority || "high",
          alert.zone || alert.location_name || "Campus",
          "received",
          alert.timestamp,
          snapshot,
          alert.location_name || null,
          alert.guard_post || null,
          alert.zone_description || null,
        ]
      );
    }
  }
}
```

```

);
console.log(`[History] ✓ Alerta guardada: ${alert.alert_id} | Zona: ${alert.zone}`);
} catch (err) {
console.error("[History] Error guardando alerta:", err.message);
await new Promise((r) => setTimeout(r, 2000));
}
}
}
}

```

```

// — Procesador de transcripciones —
async function processTranscriptionQueue() {
console.log("[History] Escuchando cola queue:transcription_save...");
while (true) {
try {
const result = await redis.brpop("queue:transcription_save", 5);
if (!result) continue;
const data = JSON.parse(result[1]);
await masterPool.query(
`UPDATE alerts SET transcription = $1 WHERE id = $2`,
[data.transcription, data.alert_id]
);
console.log(`[History] Transcripción guardada para ${data.alert_id}`);
} catch (err) {
console.error("[History] Error guardando transcripción:", err.message);
await new Promise((r) => setTimeout(r, 2000));
}
}
}
}

```

```

// — REST API —
app.get("/health", async (req, res) => {
try {

```

```

await replicaPool.query("SELECT 1");
res.json({ status: "ok", service: "history", context: "campus", db: "connected" });
} catch (e) {
res.status(500).json({ status: "error", db: e.message });
}
});

```

```

app.get("/alerts", async (req, res) => {
try {
const { from, to, zone, priority, device_id, status, location_name, guard_post, limit = 50, offset
= 0 } = req.query;
const conditions = []; const values = []; let idx = 1;

```

```

if (from) { conditions.push(`timestamp >= ${idx++}`); values.push(from); }
if (to) { conditions.push(`timestamp <= ${idx++}`); values.push(to); }
if (zone) { conditions.push(`zone ILIKE ${idx++}`); values.push(`%${zone}%`); }
if (priority) { conditions.push(`priority = ${idx++}`); values.push(priority); }
if (device_id) { conditions.push(`device_id = ${idx++}`); values.push(device_id); }
if (status) { conditions.push(`status = ${idx++}`); values.push(status); }
if (location_name) { conditions.push(`location_name ILIKE ${idx++}`);
values.push(`%${location_name}%`); }
if (guard_post) { conditions.push(`guard_post ILIKE ${idx++}`);
values.push(`%${guard_post}%`); }

```

```

const where = conditions.length ? `WHERE ${conditions.join(" AND ")}` : "";
const countResult = await replicaPool.query(`SELECT COUNT(*) FROM alerts ${where}`,
values);
const total = parseInt(countResult.rows[0].count);

```

```

const query = `
SELECT id, device_id, latitude, longitude,
emergency_type, priority, zone, status,
timestamp, created_at, snapshot_url, transcription,

```

```
location_name, guard_post, zone_description
```

```
FROM alerts ${where}
```

```
ORDER BY timestamp DESC
```

```
LIMIT ${idx++} OFFSET ${idx++}
```

```
`;
```

```
values.push(parseInt(limit)); values.push(parseInt(offset));
```

```
const result = await replicaPool.query(query, values);
```

```
res.json({ total, returned: result.rowCount, alerts: result.rows, source: "replica" });
```

```
} catch (err) {
```

```
res.status(500).json({ error: err.message });
```

```
}
```

```
});
```

```
app.patch("/alerts/:id/status", async (req, res) => {
```

```
try {
```

```
const { status, device_id } = req.body;
```

```
const allowed = ["confirmed", "false_alarm", "received", "reviewing"];
```

```
if (!allowed.includes(status))
```

```
return res.status(400).json({ error: `Status inválido. Permitidos: ${allowed.join(", ")}` });
```

```
if (status === "reviewing") {
```

```
if (device_id) notificarTotem(device_id, "reviewing");
```

```
else {
```

```
const r = await replicaPool.query("SELECT device_id FROM alerts WHERE id = $1",  
[req.params.id]);
```

```
if (r.rowCount > 0) notificarTotem(r.rows[0].device_id, "reviewing");
```

```
}
```

```
return res.json({ message: "Tótem notificado — en revisión" });
```

```
}
```

```
const result = await masterPool.query(
```

```
"UPDATE alerts SET status = $1 WHERE id = $2 RETURNING *",
```



```
[status, req.params.id]
);
```

```
if (result.rowCount === 0) {
  if (device_id) notificarTotem(device_id, status);
  return res.status(200).json({ message: "Tótem notificado (BD pendiente)" });
}
```

```
const alerta = result.rows[0];
if (alerta.device_id) notificarTotem(alerta.device_id, status);
res.json({ message: "Status actualizado", alert: alerta });
} catch (err) {
  res.status(500).json({ error: err.message });
}
});
```

```
app.get("/alerts/:id", async (req, res) => {
  try {
    const result = await replicaPool.query("SELECT * FROM alerts WHERE id = $1",
    [req.params.id]);
    if (result.rowCount === 0) return res.status(404).json({ error: "Alerta no encontrada" });
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
app.get("/stats", async (req, res) => {
  try {
    const totales = await replicaPool.query(`
    SELECT
    COUNT(*) AS total,
    COUNT(*) FILTER (WHERE priority = 'critical') AS critical,
```

```

COUNT(*) FILTER (WHERE priority = 'high') AS high,
COUNT(*) FILTER (WHERE priority = 'medium') AS medium,
COUNT(*) FILTER (WHERE status = 'confirmed') AS confirmed,
COUNT(*) FILTER (WHERE status = 'false_alarm') AS false_alarm,
COUNT(*) FILTER (WHERE status = 'received') AS pending
FROM alerts
`);

const porZona = await replicaPool.query(`
SELECT zone, location_name, guard_post, COUNT(*) AS total,
COUNT(*) FILTER (WHERE priority = 'critical') AS critical,
COUNT(*) FILTER (WHERE priority = 'high') AS high,
COUNT(*) FILTER (WHERE priority = 'medium') AS medium
FROM alerts GROUP BY zone, location_name, guard_post ORDER BY total DESC LIMIT 10
`);

const porDispositivo = await replicaPool.query(`
SELECT device_id, location_name, COUNT(*) AS total FROM alerts GROUP BY device_id,
location_name ORDER BY total DESC
`);

res.json({
  totales: totales.rows[0],
  por_zona: porZona.rows,
  por_dispositivo: porDispositivo.rows,
  source: "replica"
});

} catch (err) {
  res.status(500).json({ error: err.message });
}

});

app.listen(PORT, async () => {
  console.log(`[History] Servicio Campus iniciado en puerto ${PORT}`);
  conectarMQTT();
  await migrateDB();

```

```
processQueue();
```

```
processTranscriptionQueue();
```

```
});
```